CLASS 9 · CBSE · GANITA MANJARI PART 2 · SHORT NOTES

# Chapter 11: The World of Algorithms

Topic-wise short notes with solved examples

An algorithm is a precise step-by-step procedure for solving a problem. This chapter writes algorithms for addition and for the greatest common divisor, improves them step by step, and compares how much work they need.

## 1 What an algorithm is

- An **algorithm** is a step-by-step procedure that leads to the answer. Each step is stated so precisely that it can be followed without any further thinking.
- An algorithm is built from **basic steps** that we assume can be done directly, such as adding two single digits or checking whether one number divides another.
- A step may be **repeated** ("for each $j$ in $1, 2, \ldots, n$") or **conditional** ("if $j$ divides $n$, then ...").
- Working through an algorithm step by step is called **executing** it.
- Two questions are asked about every algorithm: Is it **correct** (why does it work)? Is it **efficient** (how many steps does it take)?

> REMEMBER
> The word comes from the name of Al-Khwārizmī (780–850 CE) of Baghdad, who wrote a book explaining the Indian place-value numerals and the Indian methods of arithmetic. In Latin his name became Algoritmi, and "algorithm" first meant the Indian way of calculating.

## 2 The addition algorithm

- **Step 1**: Write the numbers one below the other with their digits lined up from the right.
- **Step 2**: Add the rightmost digits. If the sum is less than 10, write it below and set the carry to 0. Otherwise write its units digit and set the carry to 1.
- **Step 3**: Move one column to the left. Add the two digits and the carry, and write the result in the same way.
- **Step 4**: Repeat Step 3 until no columns are left.
- **Step 5**: If the carry is 1, write 1 at the left of the answer.
- **Why it works**: each number is grouped into units, tens, hundreds and so on, and each group is added separately. Ten of one group make one of the next group, which is the carry.
- **Why it is efficient**: counting dots one by one takes 10 times as long when the numbers get one digit longer. The algorithm needs only one more column.

**Example 2.1** Execute the algorithm for $586 + 947$.

SOLUTION

Units: $6 + 7 = 13$. Write 3, carry 1.

Tens: $8 + 4 + 1 = 13$. Write 3, carry 1.

Hundreds: $5 + 9 + 1 = 15$. Write 5, carry 1.

No columns are left and the carry is 1, so write 1 on the left.

**1533**

## 3 Finding the divisors of a number

- A divisor (factor) of $n$ lies between 1 and $n$. The simplest method checks every number in that range.
- The answers are kept in a **list**, written in square brackets. A list is an example of a **data structure**: a way of organising information for an algorithm.
- **Algorithm divisors(n)**
- **Step 1**: Start with an empty list.
- **Step 2**: For each number $j$ in $1, 2, 3, \ldots, n$: if $j$ divides $n$, add $j$ at the end of the list.
- Names such as $j$ and "list of divisors" let the algorithm refer to values it is keeping track of.
- The list comes out in increasing order, because the numbers are checked in order.

**Example 3.1** Execute divisors(20).

SOLUTION

Start with [ ]. 1 divides 20: [1]. 2 divides 20: [1, 2]. 3 does not. 4 divides 20: [1, 2, 4]. 5 divides 20: [1, 2, 4, 5].

6 to 9 do not. 10 divides 20: [1, 2, 4, 5, 10]. 11 to 19 do not. 20 divides 20.

**[1, 2, 4, 5, 10, 20]**

## 4 A first algorithm for the gcd

- The **greatest common divisor** (gcd), also called the highest common factor (hcf), of two numbers is the largest number that divides both.
- **Algorithm gcd(m, n), straight from the definition**
- **Step 1**: Compute divisors($m$).
- **Step 2**: Compute divisors($n$).
- **Step 3**: Start with an empty list of common divisors.
- **Step 4**: For each $x$ in the first list: if $x$ is also in the second list, add $x$ to the list of common divisors.
- **Step 5**: Report the last (rightmost) element of the list of common divisors.
- It is correct because it follows the definition exactly. The list of common divisors is never empty, since 1 divides every number.

**Example 4.1** Use this algorithm to find gcd(20, 28).

SOLUTION

divisors(20) = [1, 2, 4, 5, 10, 20] and divisors(28) = [1, 2, 4, 7, 14, 28]

Common divisors: [1, 2, 4]

**gcd(20, 28) = 4**

## 5 Improving the algorithm

- **One scan instead of two**: check each $j$ once against both numbers. This cuts the numbers checked from $m + n$ to the larger of $m$ and $n$.
- **Stop at the smaller number**: a common divisor cannot be larger than the smaller number, so check only up to $\min(m, n)$.
- **No lists**: only the largest common divisor is wanted. Keep one value, the most recent common divisor, starting at 1, and replace it each time a larger common divisor is found.

**Example 5.1** Execute the improved algorithm for gcd(8, 20).

SOLUTION

$\min(8, 20) = 8$. Start with the most recent common divisor equal to 1.

$k = 2$ divides both: update to 2. $k = 3$ does not. $k = 4$ divides both: update to 4.

$k = 5, 6, 7$ do not divide both. $k = 8$ divides 8 but not 20.

**gcd(8, 20) = 4**

- Each improvement saves work but keeps the answer the same. Even so, about $\min(m, n)$ numbers are checked, so one more digit still means 10 times the work.

## 6 Euclid's subtraction algorithm

- **Key fact**: if $m \geq n$, then $\gcd(m, n) = \gcd(n, m - n)$.
- Reason: if $d$ divides $m$ and $n$, say $m = ad$ and $n = bd$, then $m - n = (a - b)d$, so $d$ divides $m - n$. Conversely, if $d$ divides $n$ and $m - n$, it divides their sum $m$. So the two pairs have exactly the same common divisors.
- **Algorithm**
- **Step 1**: If $m < n$, swap the two numbers.
- **Step 2**: If $n = 0$, the answer is $m$.
- **Step 3**: Otherwise replace the problem by $\gcd(n, m - n)$ and go back to Step 1.
- Each step replaces the problem by a smaller one of the same kind.

**Example 6.1** Find gcd(48, 18) by Euclid's subtraction algorithm.

SOLUTION

$\gcd(48, 18) \to \gcd(18, 30) \to$ swap $\to \gcd(30, 18) \to \gcd(18, 12) \to \gcd(12, 6) \to \gcd(6, 6) \to \gcd(6, 0)$

**gcd(48, 18) = 6**

- Weakness: for a pair like gcd(99, 2) it subtracts 2 again and again, about 50 times. The number of steps still depends on the size of the numbers.

## 7 Āryabhaṭa's division algorithm

- Subtracting $n$ from $m$ again and again until it cannot be done leaves the **remainder** when $m$ is divided by $n$. This remainder is written $m \bmod n$. For example, $17 \bmod 5 = 2$ and $33 \bmod 7 = 5$.
- So all the repeated subtractions can be done in one division: $\gcd(m, n) = \gcd(n, m \bmod n)$.
- **Algorithm**
- **Step 1**: If $m < n$, swap the two numbers.
- **Step 2**: If $n = 0$, the answer is $m$.
- **Step 3**: Otherwise replace the problem by $\gcd(n, m \bmod n)$ and go back to Step 1.
- This is the "long division" method for the gcd found in Āryabhaṭa's Āryabhaṭīya (499 CE): divide, then divide the divisor by the remainder, and continue until the remainder is 0. The last divisor is the gcd.
- The number of steps is now proportional to the number of **digits**, which makes it fast even for very large numbers.

**Example 7.1** Find gcd(252, 105) by the division algorithm.

SOLUTION

$252 \bmod 105 = 42$, so $\gcd(252, 105) = \gcd(105, 42)$

$105 \bmod 42 = 21$, so it equals $\gcd(42, 21)$

$42 \bmod 21 = 0$, so it equals $\gcd(21, 0)$

**gcd(252, 105) = 21**

**Example 7.2** Find gcd(1071, 462).

SOLUTION

$1071 = 462 \times 2 + 147$, $462 = 147 \times 3 + 21$, $147 = 21 \times 7 + 0$

**gcd(1071, 462) = 21**

**Example 7.3** Compare the two versions of Euclid's algorithm for gcd(99, 2).

SOLUTION

Subtraction: $\gcd(97, 2), \gcd(95, 2), \ldots$ and so on, about 50 reductions.

Division: $99 \bmod 2 = 1$ gives $\gcd(2, 1)$, then $2 \bmod 1 = 0$ gives $\gcd(1, 0)$.

**Division needs only 2 reductions. The answer is 1.**

## 8 Comparing the algorithms

| Algorithm | Work for gcd(m, n) |
|---|---|
| Lists of divisors | checks $m + n$ numbers |
| Single scan up to the smaller number | checks $\min(m, n)$ numbers |
| Euclid, by subtraction | can still take about $\frac{m}{n}$ steps |
| Āryabhaṭa, by division | proportional to the number of digits |

- Other algorithms can be built from these. **prime(n)**: compute divisors($n$) and report "prime" if the list has exactly two elements. **lcm(m, n)**: compute the gcd, then $\frac{m \times n}{\gcd(m, n)}$.

## 9 Common mistakes

- Writing a vague step such as "find the common divisors". Each step must say exactly what to do.
- Forgetting the last carry in the addition algorithm (Step 5).
- Stopping Euclid's algorithm when the two numbers are equal and reporting 0. Continue until the second number is 0, and report the first.
- Subtracting or dividing the wrong way round. Put the larger number first.
- Confusing $m \bmod n$ with the quotient. It is the remainder.